

Project Identity and Submission Record

KRISHIGYAAN: AI-ASSISTED MULTILINGUAL AGRICULTURAL SUPPORT PLATFORM

Submission Field	Details
Student Name	Piyush Nath
Enrollment / Roll Number	_____
University / Institution	_____
Department	Bachelor of Computer Applications
Project Guide	_____
Academic Session	_____
Date of Submission	_____

This project report is submitted in partial fulfilment of the requirements for the award of the Bachelor of Computer Applications degree. It presents the complete analysis, specification, design, implementation, testing, deployment, and evaluation of KrishiGyaan as an independent academic software project.

The submitted system addresses a practical information-access problem in agriculture by integrating secure farmer profiles, local-language interaction, weather-aware planning, government scheme support, crop and plant health analysis, and AI-assisted guidance in a single responsive application.

All blanks above are intentionally retained for institution-specific particulars. The report should be reviewed together with the executable web application, source code, deployment configuration, and viva presentation supplied as project deliverables.

The evaluation package demonstrates the complete software life cycle. Chapter-level evidence connects the stated problem to functional requirements, design diagrams, implementation modules, database behaviour, test cases, deployment controls, and identified limitations.

Assessment should consider correctness, usability, security, data validation, graceful failure, maintainability, and the student's ability to explain technical decisions. External AI and agricultural services are treated as dependencies whose availability cannot be guaranteed by the application.

Confidential deployment values are intentionally excluded. Examiners may configure authorized environment variables in a controlled environment when live integration testing is required.

The report follows an evidence chain: the problem statement establishes need; the IEEE-style SRS defines measurable behaviour; diagrams explain structure and flow; implementation chapters map those designs to code; testing chapters record acceptance conditions; and the conclusion evaluates the objectives without claiming unsupported performance.

The submitted artifacts should be treated as one controlled project version. Any later source change that alters authentication, data fields, provider contracts, language behaviour, or offline handling should trigger corresponding updates to the SRS, diagrams, test cases, report, and viva presentation.

This document intentionally uses the project as an academic case study. It avoids event, competition, award, or promotional framing and concentrates on software engineering knowledge demonstrated through a practical agricultural application.

Certificate and Supervisor Approval

This is to certify that the project entitled “KrishiGyaan: AI-Assisted Multilingual Agricultural Support Platform” is a bona fide work carried out by Piyush Nath under the supervision and guidance prescribed by the institution.

The work documented in this report includes requirements analysis, software design, database integration, implementation, testing, deployment, and evaluation. To the best of the undersigned's knowledge, the project satisfies the academic expectations of a final-year Bachelor of Computer Applications submission.

Approval Role	Name and Signature
Project Guide	_____
Head of Department	_____
External Examiner	_____
Date and Seal	_____

The certificate page is provided as a formal template. Institutional wording, seal requirements, and signatory designations may be adjusted by the department without altering the technical content of the report.

The guide's approval confirms supervision of the academic process; it does not imply that third-party AI, database, weather, crop, or plant service providers guarantee every response. The report therefore distinguishes application correctness from provider-controlled output quality.

Recommended verification includes source review, execution of the repository syntax-check command, controlled registration and login, language persistence, offline restoration, AI fallback behaviour, image-input validation, and inspection of the deployed serverless routes.

Any institution-specific corrections made after review should be recorded in the document-control section so that the submitted report, executable build, and viva presentation remain traceable to one version.

The supervisor may additionally verify that the MongoDB farmer document does not expose password values in the public profile, that duplicate mobile registration is rejected, and that provider secrets are absent from frontend files and public documentation.

Live output may vary because AI, forecast, and image-analysis services are external. Academic assessment should therefore evaluate request construction, validation, response handling, fallback, caching, and safety communication as well as the provider's sample output.

The project is suitable for demonstration on desktop and mobile browsers. The separate Android WebView package is an access wrapper around the deployed web system and does not replace the serverless backend or database architecture described in this report.

Approval signatures should be added only after the institution confirms the student's identity, project title, guide assignment, academic session, and required submission format.

Student Declaration and Originality Statement

I, Piyush Nath, declare that the project report titled “KrishiGyaan: AI-Assisted Multilingual Agricultural Support Platform” represents my academic project work and has been prepared for final-year evaluation.

Where software libraries, public APIs, documentation, standards, or external services have been used, their role has been identified in the report. No real secret key is reproduced in this document. Screens, workflows, diagrams, requirements, and test cases are described specifically for the submitted implementation.

I further declare that the project has not been presented in this report as an award entry, event submission, or commercial product claim. It is documented solely as a standalone academic software engineering project intended to demonstrate analysis, design, programming, database, networking, testing, deployment, and documentation skills.

Declaration Field	Entry
Student Signature	_____
Place	_____
Date	_____

I understand that academic evaluation may include a source-code walkthrough, database inspection, test repetition, diagram interpretation, and questions about implementation trade-offs. I accept responsibility for explaining the submitted design and its limitations.

I confirm that API keys, database passwords, and other operational secrets must remain in protected environment configuration and must not be included in public repositories, screenshots, report appendices, or presentation notes.

I also acknowledge that AI-generated agricultural guidance is decision support. Users should verify high-risk crop-treatment, chemical-application, financial, or eligibility decisions with qualified agricultural officers and official scheme sources.

All diagrams in the report are logical representations of the submitted implementation. The MongoDB entity model distinguishes the physically stored farmer collection from logical browser-side records used for sessions, drafts, health results, and offline snapshots.

Testing status is reported honestly. The repository contains a JavaScript syntax-check command and the report defines repeatable manual acceptance cases; it does not claim that a dedicated automated unit-test framework has already been implemented.

Any similarity to public technical terminology, framework names, API names, or software engineering standards reflects necessary reference to established technologies. Such references do not transfer ownership of those services or documentation.

The final project should be operated with lawful, authorized credentials and a controlled database account. Personal farmer information should be collected only for the stated advisory and authentication purpose.

Acknowledgement

Acknowledgement is examined from the perspective of academic purpose. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Acknowledgement is examined from the perspective of system coverage. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Acknowledgement is examined from the perspective of implementation evidence. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Acknowledgement is examined from the perspective of evaluation method. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Acknowledgement is examined from the perspective of document organization. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Abstract

Abstract is examined from the perspective of academic purpose. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Abstract is examined from the perspective of system coverage. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Abstract is examined from the perspective of implementation evidence. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Abstract is examined from the perspective of evaluation method. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Abstract is examined from the perspective of document organization. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Executive Summary

Executive Summary is examined from the perspective of academic purpose. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Executive Summary is examined from the perspective of system coverage. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Executive Summary is examined from the perspective of implementation evidence. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Executive Summary is examined from the perspective of evaluation method. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Executive Summary is examined from the perspective of document organization. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Table of Contents and Report Map

Table of Contents and Report Map is examined from the perspective of academic purpose. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Table of Contents and Report Map is examined from the perspective of system coverage. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Table of Contents and Report Map is examined from the perspective of implementation evidence. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Table of Contents and Report Map is examined from the perspective of evaluation method. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Table of Contents and Report Map is examined from the perspective of document organization. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

List of Figures, Tables and Abbreviations

List of Figures, Tables and Abbreviations is examined from the perspective of academic purpose. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

List of Figures, Tables and Abbreviations is examined from the perspective of system coverage. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

List of Figures, Tables and Abbreviations is examined from the perspective of implementation evidence. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

List of Figures, Tables and Abbreviations is examined from the perspective of evaluation method. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

List of Figures, Tables and Abbreviations is examined from the perspective of document organization. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Chapter 1: Introduction

Chapter 1: Introduction is examined from the perspective of problem context and academic relevance. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Chapter 1: Introduction is examined from the perspective of implemented behaviour and design evidence. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Chapter 1: Introduction is examined from the perspective of constraints, assumptions, and trade-offs. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Chapter 1: Introduction is examined from the perspective of measurement, acceptance, and future refinement. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Background of Digital Agriculture

Background of Digital Agriculture is examined from the perspective of problem context and academic relevance. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Background of Digital Agriculture is examined from the perspective of implemented behaviour and design evidence. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Background of Digital Agriculture is examined from the perspective of constraints, assumptions, and trade-offs. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Background of Digital Agriculture is examined from the perspective of measurement, acceptance, and future refinement. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Problem Statement

Problem Statement is examined from the perspective of problem context and academic relevance. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Problem Statement is examined from the perspective of implemented behaviour and design evidence. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Problem Statement is examined from the perspective of constraints, assumptions, and trade-offs. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Problem Statement is examined from the perspective of measurement, acceptance, and future refinement. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Project Aim and Objectives

Project Aim and Objectives is examined from the perspective of problem context and academic relevance. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Project Aim and Objectives is examined from the perspective of implemented behaviour and design evidence. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Project Aim and Objectives is examined from the perspective of constraints, assumptions, and trade-offs. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Project Aim and Objectives is examined from the perspective of measurement, acceptance, and future refinement. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Scope and Boundaries

Scope and Boundaries is examined from the perspective of problem context and academic relevance. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Scope and Boundaries is examined from the perspective of implemented behaviour and design evidence. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Scope and Boundaries is examined from the perspective of constraints, assumptions, and trade-offs. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Scope and Boundaries is examined from the perspective of measurement, acceptance, and future refinement. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Stakeholders and User Context

Stakeholders and User Context is examined from the perspective of problem context and academic relevance. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Stakeholders and User Context is examined from the perspective of implemented behaviour and design evidence. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Stakeholders and User Context is examined from the perspective of constraints, assumptions, and trade-offs. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Stakeholders and User Context is examined from the perspective of measurement, acceptance, and future refinement. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Need and Significance

Need and Significance is examined from the perspective of problem context and academic relevance. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Need and Significance is examined from the perspective of implemented behaviour and design evidence. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Need and Significance is examined from the perspective of constraints, assumptions, and trade-offs. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Need and Significance is examined from the perspective of measurement, acceptance, and future refinement. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Existing System Study

Existing System Study is examined from the perspective of problem context and academic relevance. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Existing System Study is examined from the perspective of implemented behaviour and design evidence. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Existing System Study is examined from the perspective of constraints, assumptions, and trade-offs. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Existing System Study is examined from the perspective of measurement, acceptance, and future refinement. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Limitations of Existing Approaches

Limitations of Existing Approaches is examined from the perspective of problem context and academic relevance. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Limitations of Existing Approaches is examined from the perspective of implemented behaviour and design evidence. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Limitations of Existing Approaches is examined from the perspective of constraints, assumptions, and trade-offs. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Limitations of Existing Approaches is examined from the perspective of measurement, acceptance, and future refinement. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Proposed System Overview

Proposed System Overview is examined from the perspective of problem context and academic relevance. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Proposed System Overview is examined from the perspective of implemented behaviour and design evidence. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Proposed System Overview is examined from the perspective of constraints, assumptions, and trade-offs. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Proposed System Overview is examined from the perspective of measurement, acceptance, and future refinement. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Comparative Analysis

Comparative Analysis is examined from the perspective of problem context and academic relevance. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Comparative Analysis is examined from the perspective of implemented behaviour and design evidence. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Comparative Analysis is examined from the perspective of constraints, assumptions, and trade-offs. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Comparative Analysis is examined from the perspective of measurement, acceptance, and future refinement. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Technical

Feasibility Study: Technical is examined from the perspective of problem context and academic relevance. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Technical is examined from the perspective of implemented behaviour and design evidence. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Technical is examined from the perspective of constraints, assumptions, and trade-offs. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Feasibility Study: Technical is examined from the perspective of measurement, acceptance, and future refinement. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Operational

Feasibility Study: Operational is examined from the perspective of problem context and academic relevance. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Operational is examined from the perspective of implemented behaviour and design evidence. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Operational is examined from the perspective of constraints, assumptions, and trade-offs. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Feasibility Study: Operational is examined from the perspective of measurement, acceptance, and future refinement. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Economic and Schedule

Feasibility Study: Economic and Schedule is examined from the perspective of problem context and academic relevance. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Economic and Schedule is examined from the perspective of implemented behaviour and design evidence. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Feasibility Study: Economic and Schedule is examined from the perspective of constraints, assumptions, and trade-offs. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Feasibility Study: Economic and Schedule is examined from the perspective of measurement, acceptance, and future refinement. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Risk Assessment

Risk Assessment is examined from the perspective of problem context and academic relevance. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Risk Assessment is examined from the perspective of implemented behaviour and design evidence. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Risk Assessment is examined from the perspective of constraints, assumptions, and trade-offs. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Risk Assessment is examined from the perspective of measurement, acceptance, and future refinement. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Development Methodology and SDLC

Development Methodology and SDLC is examined from the perspective of problem context and academic relevance. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Development Methodology and SDLC is examined from the perspective of implemented behaviour and design evidence. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Development Methodology and SDLC is examined from the perspective of constraints, assumptions, and trade-offs. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Development Methodology and SDLC is examined from the perspective of measurement, acceptance, and future refinement. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Project Planning and Milestones

Project Planning and Milestones is examined from the perspective of problem context and academic relevance. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Project Planning and Milestones is examined from the perspective of implemented behaviour and design evidence. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Project Planning and Milestones is examined from the perspective of constraints, assumptions, and trade-offs. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Project Planning and Milestones is examined from the perspective of measurement, acceptance, and future refinement. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: Document Control

IEEE SRS: Document Control is examined from the perspective of IEEE-style requirement specification. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-001	The system shall support the behaviour defined under ieee srs: document control with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-001	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: Document Control is examined from the perspective of preconditions and trigger. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: Document Control is examined from the perspective of normal flow and postconditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: Document Control is examined from the perspective of exception flow. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: Document Control is examined from the perspective of quality attributes. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: Purpose, Scope and Audience

IEEE SRS: Purpose, Scope and Audience is examined from the perspective of IEEE-style requirement specification. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-002	The system shall support the behaviour defined under ieee srs: purpose, scope and audience with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-002	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: Purpose, Scope and Audience is examined from the perspective of preconditions and trigger. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: Purpose, Scope and Audience is examined from the perspective of normal flow and postconditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: Purpose, Scope and Audience is examined from the perspective of exception flow. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: Purpose, Scope and Audience is examined from the perspective of quality attributes. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: Definitions and References

IEEE SRS: Definitions and References is examined from the perspective of IEEE-style requirement specification. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-003	The system shall support the behaviour defined under ieee srs: definitions and references with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-003	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: Definitions and References is examined from the perspective of preconditions and trigger. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: Definitions and References is examined from the perspective of normal flow and postconditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: Definitions and References is examined from the perspective of exception flow. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: Definitions and References is examined from the perspective of quality attributes. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: Product Perspective

IEEE SRS: Product Perspective is examined from the perspective of IEEE-style requirement specification. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-004	The system shall support the behaviour defined under ieee srs: product perspective with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-004	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: Product Perspective is examined from the perspective of preconditions and trigger. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: Product Perspective is examined from the perspective of normal flow and postconditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: Product Perspective is examined from the perspective of exception flow. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: Product Perspective is examined from the perspective of quality attributes. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: Product Functions

IEEE SRS: Product Functions is examined from the perspective of IEEE-style requirement specification. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-005	The system shall support the behaviour defined under ieee srs: product functions with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-005	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: Product Functions is examined from the perspective of preconditions and trigger. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: Product Functions is examined from the perspective of normal flow and postconditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: Product Functions is examined from the perspective of exception flow. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: Product Functions is examined from the perspective of quality attributes. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: User Classes

IEEE SRS: User Classes is examined from the perspective of IEEE-style requirement specification. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-006	The system shall support the behaviour defined under ieee srs: user classes with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-006	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: User Classes is examined from the perspective of preconditions and trigger. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: User Classes is examined from the perspective of normal flow and postconditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: User Classes is examined from the perspective of exception flow. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: User Classes is examined from the perspective of quality attributes. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: Operating Environment

IEEE SRS: Operating Environment is examined from the perspective of IEEE-style requirement specification. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-007	The system shall support the behaviour defined under ieee srs: operating environment with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-007	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: Operating Environment is examined from the perspective of preconditions and trigger. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: Operating Environment is examined from the perspective of normal flow and postconditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: Operating Environment is examined from the perspective of exception flow. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: Operating Environment is examined from the perspective of quality attributes. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

IEEE SRS: Constraints and Dependencies

IEEE SRS: Constraints and Dependencies is examined from the perspective of IEEE-style requirement specification. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-008	The system shall support the behaviour defined under ieee srs: constraints and dependencies with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-008	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

IEEE SRS: Constraints and Dependencies is examined from the perspective of preconditions and trigger. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

IEEE SRS: Constraints and Dependencies is examined from the perspective of normal flow and postconditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

IEEE SRS: Constraints and Dependencies is examined from the perspective of exception flow. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

IEEE SRS: Constraints and Dependencies is examined from the perspective of quality attributes. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Registration

Functional Requirements: Registration is examined from the perspective of IEEE-style requirement specification. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-009	The system shall support the behaviour defined under functional requirements: registration with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-009	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Registration is examined from the perspective of preconditions and trigger. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Registration is examined from the perspective of normal flow and postconditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Registration is examined from the perspective of exception flow. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Registration is examined from the perspective of quality attributes. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Authentication

Functional Requirements: Authentication is examined from the perspective of IEEE-style requirement specification. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-010	The system shall support the behaviour defined under functional requirements: authentication with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-010	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Authentication is examined from the perspective of preconditions and trigger. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Authentication is examined from the perspective of normal flow and postconditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Authentication is examined from the perspective of exception flow. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Authentication is examined from the perspective of quality attributes. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Profile and Session

Functional Requirements: Profile and Session is examined from the perspective of IEEE-style requirement specification. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-011	The system shall support the behaviour defined under functional requirements: profile and session with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-011	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Profile and Session is examined from the perspective of preconditions and trigger. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Profile and Session is examined from the perspective of normal flow and postconditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Profile and Session is examined from the perspective of exception flow. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Profile and Session is examined from the perspective of quality attributes. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Language and Voice

Functional Requirements: Language and Voice is examined from the perspective of IEEE-style requirement specification. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-012	The system shall support the behaviour defined under functional requirements: language and voice with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-012	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Language and Voice is examined from the perspective of preconditions and trigger. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Language and Voice is examined from the perspective of normal flow and postconditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Language and Voice is examined from the perspective of exception flow. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Language and Voice is examined from the perspective of quality attributes. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: AI Assistant

Functional Requirements: AI Assistant is examined from the perspective of IEEE-style requirement specification. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-013	The system shall support the behaviour defined under functional requirements: ai assistant with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-013	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: AI Assistant is examined from the perspective of preconditions and trigger. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: AI Assistant is examined from the perspective of normal flow and postconditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: AI Assistant is examined from the perspective of exception flow. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: AI Assistant is examined from the perspective of quality attributes. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Scheme Discovery

Functional Requirements: Scheme Discovery is examined from the perspective of IEEE-style requirement specification. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-014	The system shall support the behaviour defined under functional requirements: scheme discovery with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-014	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Scheme Discovery is examined from the perspective of preconditions and trigger. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Scheme Discovery is examined from the perspective of normal flow and postconditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Scheme Discovery is examined from the perspective of exception flow. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Scheme Discovery is examined from the perspective of quality attributes. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Document Drafting

Functional Requirements: Document Drafting is examined from the perspective of IEEE-style requirement specification. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-015	The system shall support the behaviour defined under functional requirements: document drafting with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-015	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Document Drafting is examined from the perspective of preconditions and trigger. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Document Drafting is examined from the perspective of normal flow and postconditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Document Drafting is examined from the perspective of exception flow. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Document Drafting is examined from the perspective of quality attributes. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Crop Health

Functional Requirements: Crop Health is examined from the perspective of IEEE-style requirement specification. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-016	The system shall support the behaviour defined under functional requirements: crop health with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-016	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Crop Health is examined from the perspective of preconditions and trigger. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Crop Health is examined from the perspective of normal flow and postconditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Crop Health is examined from the perspective of exception flow. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Crop Health is examined from the perspective of quality attributes. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Plant Health

Functional Requirements: Plant Health is examined from the perspective of IEEE-style requirement specification. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-017	The system shall support the behaviour defined under functional requirements: plant health with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-017	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Plant Health is examined from the perspective of preconditions and trigger. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Plant Health is examined from the perspective of normal flow and postconditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Plant Health is examined from the perspective of exception flow. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Plant Health is examined from the perspective of quality attributes. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Soil Guidance

Functional Requirements: Soil Guidance is examined from the perspective of IEEE-style requirement specification. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-018	The system shall support the behaviour defined under functional requirements: soil guidance with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-018	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Soil Guidance is examined from the perspective of preconditions and trigger. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Soil Guidance is examined from the perspective of normal flow and postconditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Soil Guidance is examined from the perspective of exception flow. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Soil Guidance is examined from the perspective of quality attributes. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Weather Advisory

Functional Requirements: Weather Advisory is examined from the perspective of IEEE-style requirement specification. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-019	The system shall support the behaviour defined under functional requirements: weather advisory with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-019	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Weather Advisory is examined from the perspective of preconditions and trigger. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Weather Advisory is examined from the perspective of normal flow and postconditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Weather Advisory is examined from the perspective of exception flow. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Weather Advisory is examined from the perspective of quality attributes. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Functional Requirements: Offline Operation

Functional Requirements: Offline Operation is examined from the perspective of IEEE-style requirement specification. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-020	The system shall support the behaviour defined under functional requirements: offline operation with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-020	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Functional Requirements: Offline Operation is examined from the perspective of preconditions and trigger. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Functional Requirements: Offline Operation is examined from the perspective of normal flow and postconditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Functional Requirements: Offline Operation is examined from the perspective of exception flow. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Functional Requirements: Offline Operation is examined from the perspective of quality attributes. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

External Interface Requirements

External Interface Requirements is examined from the perspective of IEEE-style requirement specification. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-021	The system shall support the behaviour defined under external interface requirements with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-021	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

External Interface Requirements is examined from the perspective of preconditions and trigger. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

External Interface Requirements is examined from the perspective of normal flow and postconditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

External Interface Requirements is examined from the perspective of exception flow. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

External Interface Requirements is examined from the perspective of quality attributes. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Performance Requirements

Performance Requirements is examined from the perspective of IEEE-style requirement specification. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-022	The system shall support the behaviour defined under performance requirements with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-022	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Performance Requirements is examined from the perspective of preconditions and trigger. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Performance Requirements is examined from the perspective of normal flow and postconditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Performance Requirements is examined from the perspective of exception flow. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Performance Requirements is examined from the perspective of quality attributes. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Security and Privacy Requirements

Security and Privacy Requirements is examined from the perspective of IEEE-style requirement specification. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-023	The system shall support the behaviour defined under security and privacy requirements with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-023	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Security and Privacy Requirements is examined from the perspective of preconditions and trigger. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Security and Privacy Requirements is examined from the perspective of normal flow and postconditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Security and Privacy Requirements is examined from the perspective of exception flow. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Security and Privacy Requirements is examined from the perspective of quality attributes. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Reliability, Availability and Maintainability

Reliability, Availability and Maintainability is examined from the perspective of IEEE-style requirement specification. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-024	The system shall support the behaviour defined under reliability, availability and maintainability with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-024	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Reliability, Availability and Maintainability is examined from the perspective of preconditions and trigger. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Reliability, Availability and Maintainability is examined from the perspective of normal flow and postconditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Reliability, Availability and Maintainability is examined from the perspective of exception flow. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Reliability, Availability and Maintainability is examined from the perspective of quality attributes. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Usability and Accessibility Requirements

Usability and Accessibility Requirements is examined from the perspective of IEEE-style requirement specification. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-025	The system shall support the behaviour defined under usability and accessibility requirements with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-025	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Usability and Accessibility Requirements is examined from the perspective of preconditions and trigger. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Usability and Accessibility Requirements is examined from the perspective of normal flow and postconditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

Usability and Accessibility Requirements is examined from the perspective of exception flow. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Usability and Accessibility Requirements is examined from the perspective of quality attributes. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirements Traceability Matrix

Requirements Traceability Matrix is examined from the perspective of IEEE-style requirement specification. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Requirement ID	Normative Statement	Acceptance Evidence
FR-026	The system shall support the behaviour defined under requirements traceability matrix with validated input, explicit errors, and a user-readable result.	Functional execution and response inspection
NFR-026	The system shall preserve confidentiality of provider credentials and shall degrade safely when an external dependency is unavailable.	Code inspection, failure-path test, environment review

Preconditions And Trigger

Requirements Traceability Matrix is examined from the perspective of preconditions and trigger. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Normal Flow And Postconditions

Requirements Traceability Matrix is examined from the perspective of normal flow and postconditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Exception Flow

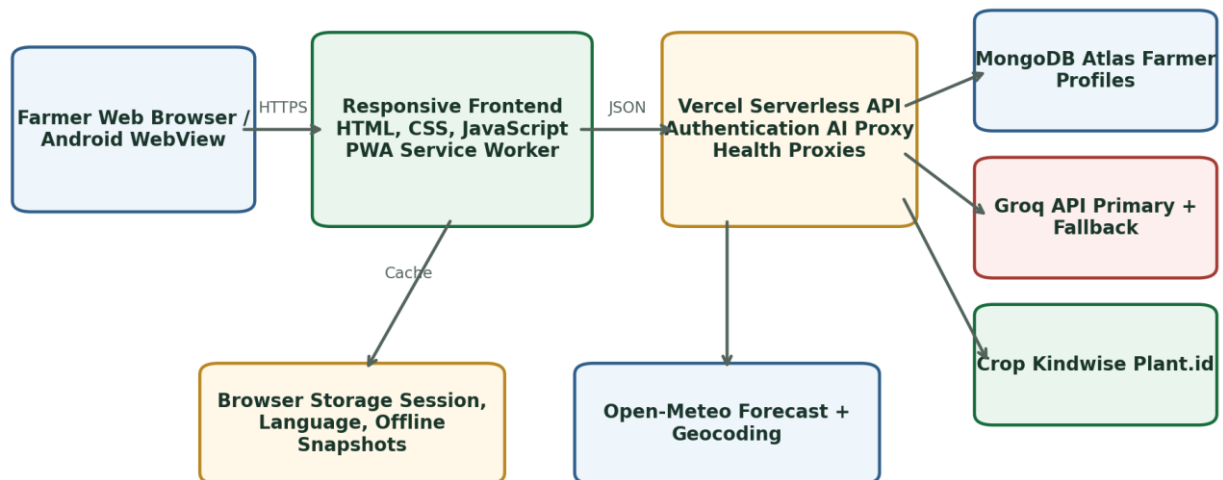
Requirements Traceability Matrix is examined from the perspective of exception flow. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Quality Attributes

Requirements Traceability Matrix is examined from the perspective of quality attributes. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

System Architecture

KrishiGyaan System Architecture



System Architecture is examined from the perspective of structural meaning and boundary definition. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

System Architecture is examined from the perspective of data ownership, actor responsibility, and verification. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Component Design

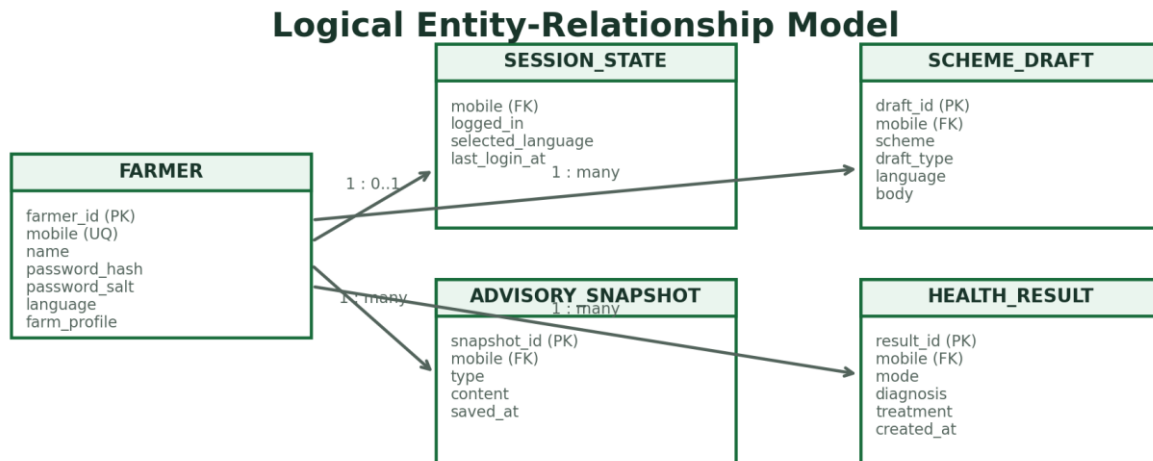
Component Design is examined from the perspective of module responsibility and implementation rationale. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Component Design is examined from the perspective of inputs, processing rules, outputs, and error handling. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Component Design is examined from the perspective of maintainability, security, and user experience. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ER Diagram



Physical MongoDB implementation stores FARMER as the persistent collection. Other entities are logical browser-side records used for traceability and offline continuity.

ER Diagram is examined from the perspective of structural meaning and boundary definition. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ER Diagram is examined from the perspective of data ownership, actor responsibility, and verification. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Database and Document Schema

Database and Document Schema is examined from the perspective of module responsibility and implementation rationale. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Database and Document Schema is examined from the perspective of inputs, processing rules, outputs, and error handling. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Database and Document Schema is examined from the perspective of maintainability, security, and user experience. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Data Dictionary: Farmer Identity

Data Dictionary: Farmer Identity is examined from the perspective of module responsibility and implementation rationale. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Data Dictionary: Farmer Identity is examined from the perspective of inputs, processing rules, outputs, and error handling. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Data Dictionary: Farmer Identity is examined from the perspective of maintainability, security, and user experience. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Data Dictionary: Farm Profile

Data Dictionary: Farm Profile is examined from the perspective of module responsibility and implementation rationale. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

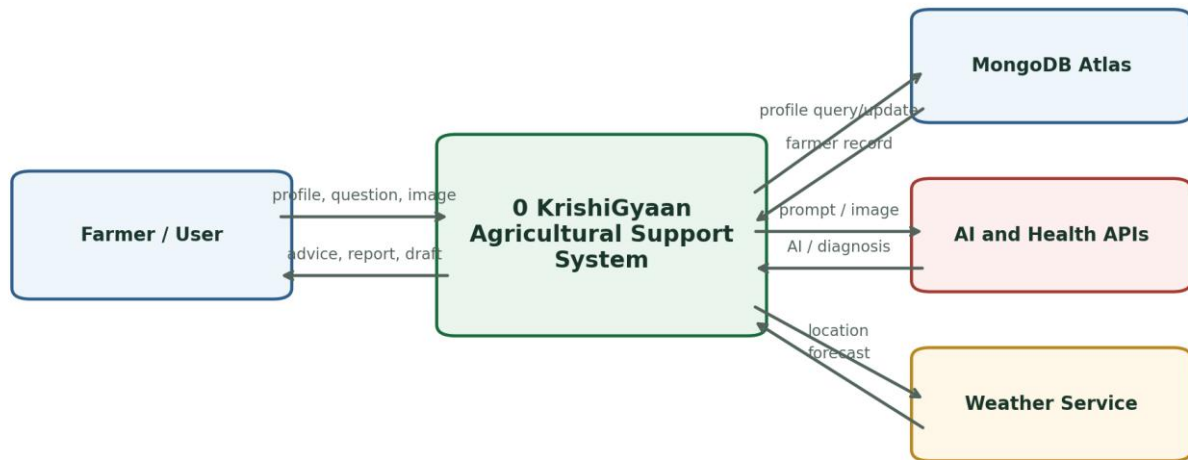
Data Dictionary: Farm Profile is examined from the perspective of inputs, processing rules, outputs, and error handling. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Data Dictionary: Farm Profile is examined from the perspective of maintainability, security, and user experience. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Data Flow Diagram Level 0

Data Flow Diagram - Level 0 (Context)



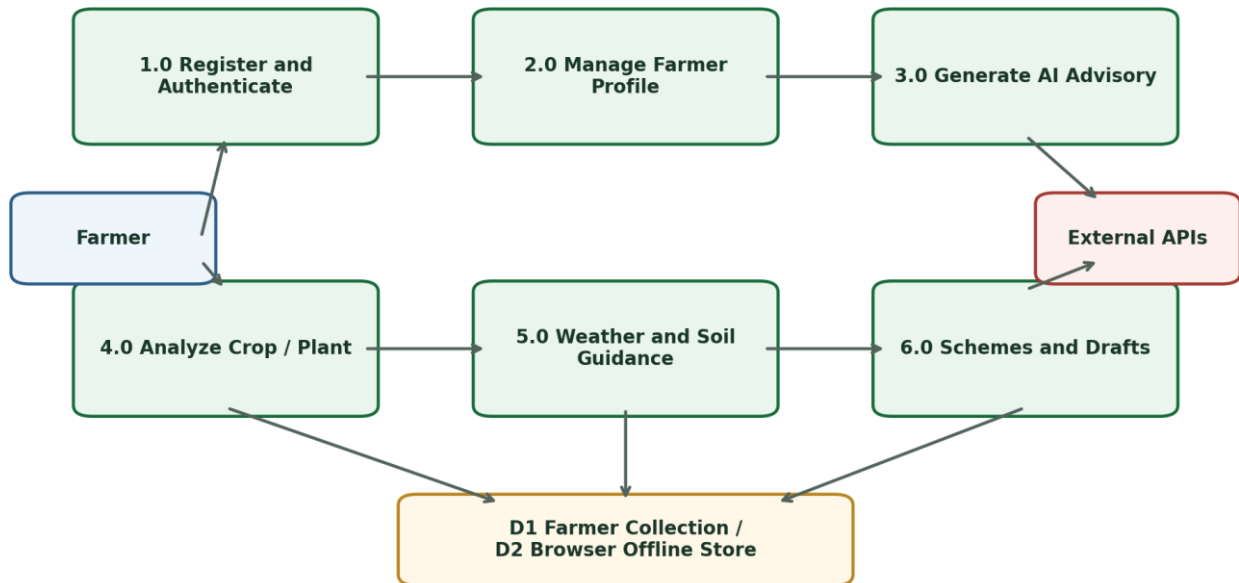
Data Flow Diagram Level 0 is examined from the perspective of structural meaning and boundary definition. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Data Flow Diagram Level 0 is examined from the perspective of data ownership, actor responsibility, and verification. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Data Flow Diagram Level 1

Data Flow Diagram - Level 1

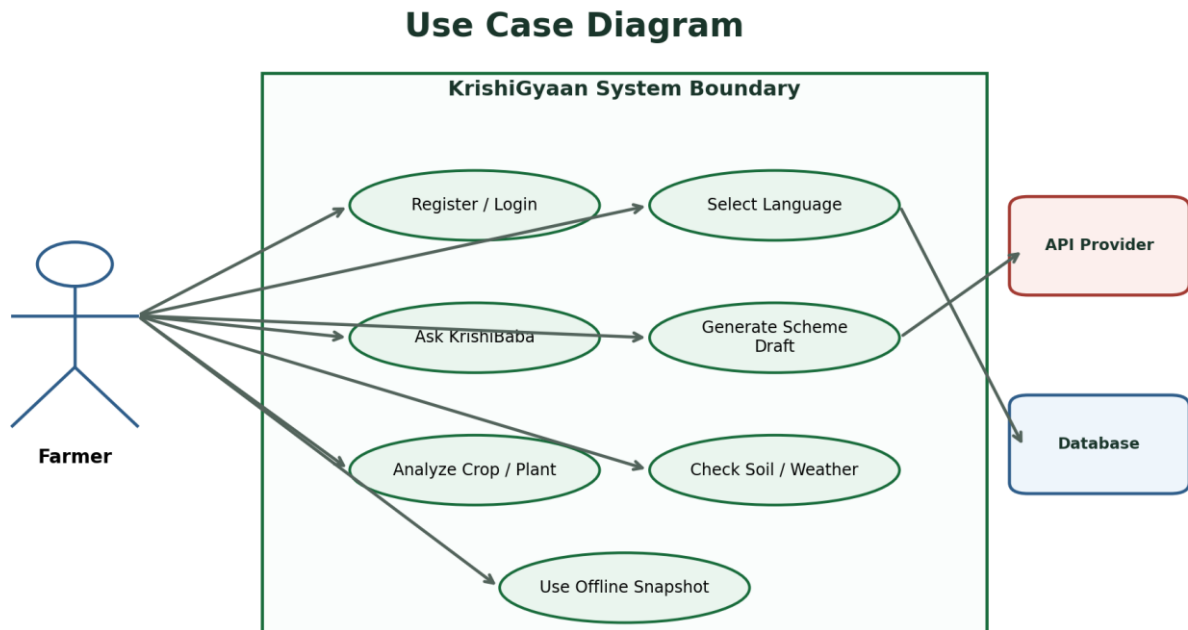


Data Flow Diagram Level 1 is examined from the perspective of structural meaning and boundary definition. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Data Flow Diagram Level 1 is examined from the perspective of data ownership, actor responsibility, and verification. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Use Case Diagram



Use Case Diagram is examined from the perspective of structural meaning and boundary definition. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Use Case Diagram is examined from the perspective of data ownership, actor responsibility, and verification. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Use Case Specifications

Use Case Specifications is examined from the perspective of module responsibility and implementation rationale. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

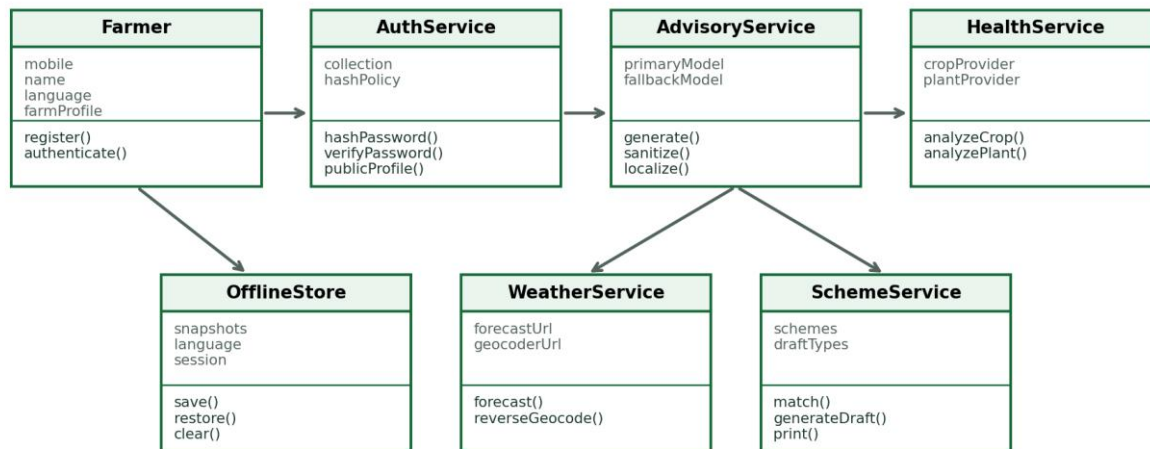
Use Case Specifications is examined from the perspective of inputs, processing rules, outputs, and error handling. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Use Case Specifications is examined from the perspective of maintainability, security, and user experience. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Class Diagram

Conceptual Class Diagram



Class Diagram is examined from the perspective of structural meaning and boundary definition. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Class Diagram is examined from the perspective of data ownership, actor responsibility, and verification. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Class Responsibilities and Collaborations

Class Responsibilities and Collaborations is examined from the perspective of module responsibility and implementation rationale. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

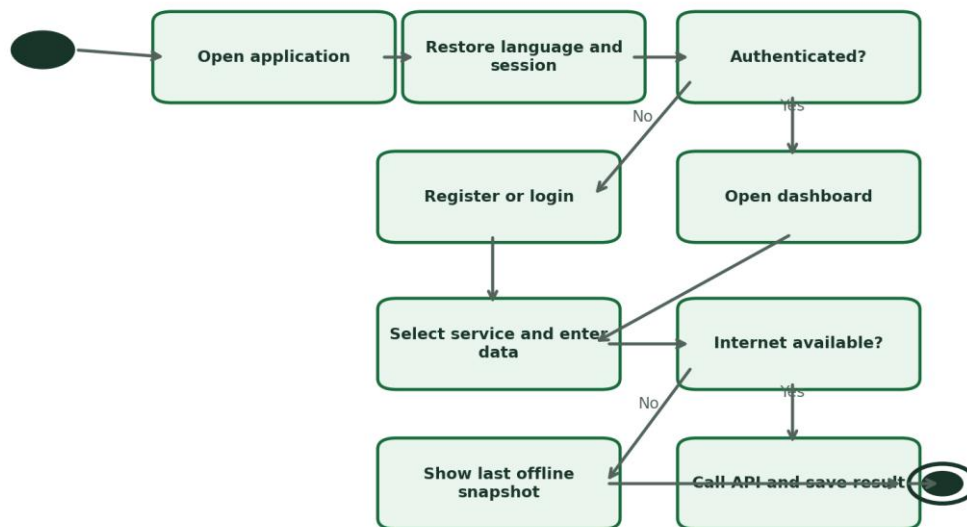
Class Responsibilities and Collaborations is examined from the perspective of inputs, processing rules, outputs, and error handling. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Class Responsibilities and Collaborations is examined from the perspective of maintainability, security, and user experience. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Activity Diagram

Activity Diagram - Farmer Advisory Workflow



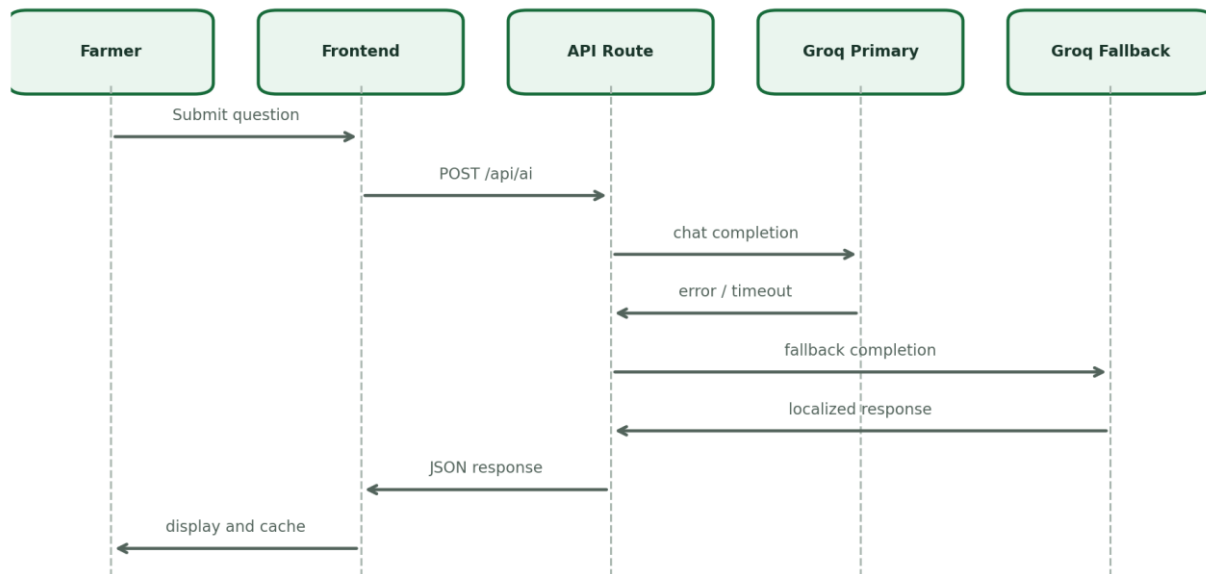
Activity Diagram is examined from the perspective of structural meaning and boundary definition. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Activity Diagram is examined from the perspective of data ownership, actor responsibility, and verification. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Sequence Diagram

Sequence Diagram - AI Advisory with Fallback



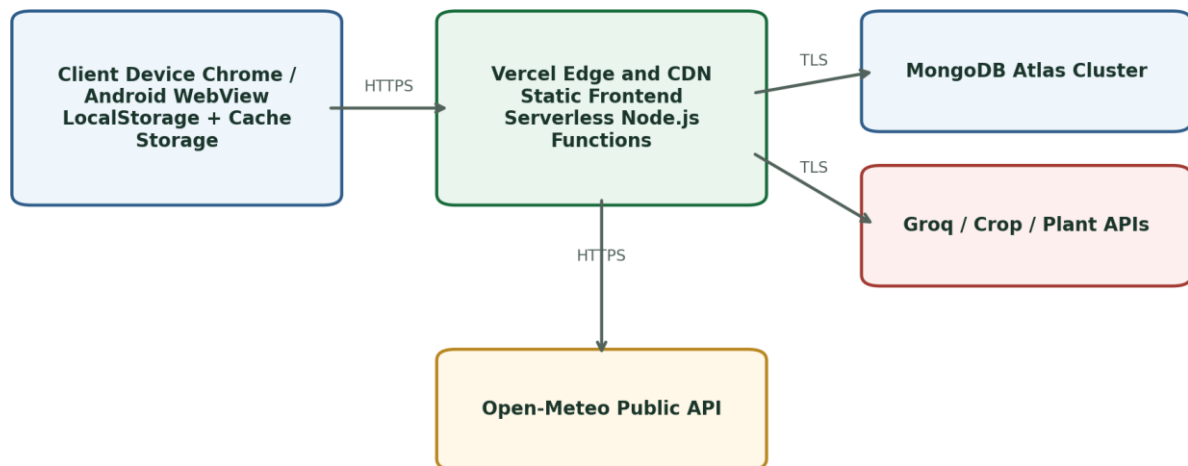
Sequence Diagram is examined from the perspective of structural meaning and boundary definition. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Sequence Diagram is examined from the perspective of data ownership, actor responsibility, and verification. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

Deployment Diagram

Deployment Diagram



Deployment Diagram is examined from the perspective of structural meaning and boundary definition. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Deployment Diagram is examined from the perspective of data ownership, actor responsibility, and verification. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Interpretation note: connectors indicate logical communication or dependency, not unrestricted access. Secret-bearing requests terminate at server-side API routes; browser storage contains user-facing session and snapshot data but not third-party provider credentials.

User Interface Design Principles

User Interface Design Principles is examined from the perspective of module responsibility and implementation rationale. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

User Interface Design Principles is examined from the perspective of inputs, processing rules, outputs, and error handling. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

User Interface Design Principles is examined from the perspective of maintainability, security, and user experience. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Home Page and Navigation

Home Page and Navigation is examined from the perspective of module responsibility and implementation rationale. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Home Page and Navigation is examined from the perspective of inputs, processing rules, outputs, and error handling. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Home Page and Navigation is examined from the perspective of maintainability, security, and user experience. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Registration Module

Registration Module is examined from the perspective of module responsibility and implementation rationale. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Registration Module is examined from the perspective of inputs, processing rules, outputs, and error handling. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Registration Module is examined from the perspective of maintainability, security, and user experience. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Login and Password Recovery Module

Login and Password Recovery Module is examined from the perspective of module responsibility and implementation rationale. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Login and Password Recovery Module is examined from the perspective of inputs, processing rules, outputs, and error handling. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Login and Password Recovery Module is examined from the perspective of maintainability, security, and user experience. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dashboard and Protected Routing

Dashboard and Protected Routing is examined from the perspective of module responsibility and implementation rationale. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dashboard and Protected Routing is examined from the perspective of inputs, processing rules, outputs, and error handling. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Dashboard and Protected Routing is examined from the perspective of maintainability, security, and user experience. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Multilingual Localization Module

Multilingual Localization Module is examined from the perspective of module responsibility and implementation rationale. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Multilingual Localization Module is examined from the perspective of inputs, processing rules, outputs, and error handling. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Multilingual Localization Module is examined from the perspective of maintainability, security, and user experience. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Speech-to-Text and Text-to-Speech

Speech-to-Text and Text-to-Speech is examined from the perspective of module responsibility and implementation rationale. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Speech-to-Text and Text-to-Speech is examined from the perspective of inputs, processing rules, outputs, and error handling. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Speech-to-Text and Text-to-Speech is examined from the perspective of maintainability, security, and user experience. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

KrishiBaba AI Integration

KrishiBaba AI Integration is examined from the perspective of module responsibility and implementation rationale. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

KrishiBaba AI Integration is examined from the perspective of inputs, processing rules, outputs, and error handling. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

KrishiBaba AI Integration is examined from the perspective of maintainability, security, and user experience. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Government Scheme Module

Government Scheme Module is examined from the perspective of module responsibility and implementation rationale. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Government Scheme Module is examined from the perspective of inputs, processing rules, outputs, and error handling. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Government Scheme Module is examined from the perspective of maintainability, security, and user experience. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Application Draft Generation

Application Draft Generation is examined from the perspective of module responsibility and implementation rationale. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Application Draft Generation is examined from the perspective of inputs, processing rules, outputs, and error handling. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Application Draft Generation is examined from the perspective of maintainability, security, and user experience. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Crop and Plant Analysis Module

Crop and Plant Analysis Module is examined from the perspective of module responsibility and implementation rationale. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Crop and Plant Analysis Module is examined from the perspective of inputs, processing rules, outputs, and error handling. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Crop and Plant Analysis Module is examined from the perspective of maintainability, security, and user experience. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Soil Health Module

Soil Health Module is examined from the perspective of module responsibility and implementation rationale. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Soil Health Module is examined from the perspective of inputs, processing rules, outputs, and error handling. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Soil Health Module is examined from the perspective of maintainability, security, and user experience. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Weather and Growth Advisory Module

Weather and Growth Advisory Module is examined from the perspective of module responsibility and implementation rationale. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Weather and Growth Advisory Module is examined from the perspective of inputs, processing rules, outputs, and error handling. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Weather and Growth Advisory Module is examined from the perspective of maintainability, security, and user experience. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Offline PWA and Recovery Workflow

Offline PWA and Recovery Workflow is examined from the perspective of module responsibility and implementation rationale. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Offline PWA and Recovery Workflow is examined from the perspective of inputs, processing rules, outputs, and error handling. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Offline PWA and Recovery Workflow is examined from the perspective of maintainability, security, and user experience. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

MongoDB Integration

MongoDB Integration is examined from the perspective of module responsibility and implementation rationale. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

MongoDB Integration is examined from the perspective of inputs, processing rules, outputs, and error handling. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

MongoDB Integration is examined from the perspective of maintainability, security, and user experience. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Serverless API Design

Serverless API Design is examined from the perspective of module responsibility and implementation rationale. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Serverless API Design is examined from the perspective of inputs, processing rules, outputs, and error handling. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Serverless API Design is examined from the perspective of maintainability, security, and user experience. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Configuration and Secret Management

Configuration and Secret Management is examined from the perspective of module responsibility and implementation rationale. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Configuration and Secret Management is examined from the perspective of inputs, processing rules, outputs, and error handling. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Configuration and Secret Management is examined from the perspective of maintainability, security, and user experience. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Vercel Deployment Procedure

Vercel Deployment Procedure is examined from the perspective of module responsibility and implementation rationale. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Vercel Deployment Procedure is examined from the perspective of inputs, processing rules, outputs, and error handling. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Vercel Deployment Procedure is examined from the perspective of maintainability, security, and user experience. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Android WebView Packaging

Android WebView Packaging is examined from the perspective of module responsibility and implementation rationale. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Android WebView Packaging is examined from the perspective of inputs, processing rules, outputs, and error handling. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Design Concern	Implemented Decision	Reason
Interface	Responsive browser-first controls with persistent language selection	Works across desktop, mobile browser, and WebView
Data	MongoDB for farmer records; browser cache for last-viewed advisory	Separates durable identity from offline convenience
Integration	Server-side proxy routes for credential-bearing providers	Avoids exposing secrets in client code
Failure	Readable error plus cached result where meaningful	Supports intermittent rural connectivity

Android WebView Packaging is examined from the perspective of maintainability, security, and user experience. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Testing Strategy and Environment

Testing Strategy and Environment is examined from the perspective of verification planning and observable acceptance criteria. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-02	Reject invalid mobile	Nine digits or invalid prefix	Validation error; no insert	Pass
TC-03	Reject weak password	No letter or fewer than 8 chars	Password policy message	Pass
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Testing Strategy and Environment is examined from the perspective of defect prevention, regression risk, and repeatability. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Unit-Level and Validation Test Cases

Unit-Level and Validation Test Cases is examined from the perspective of verification planning and observable acceptance criteria. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-03	Reject weak password	No letter or fewer than 8 chars	Password policy message	Pass
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Unit-Level and Validation Test Cases is examined from the perspective of defect prevention, regression risk, and repeatability. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Authentication Integration Test Cases

Authentication Integration Test Cases is examined from the perspective of verification planning and observable acceptance criteria. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass
TC-08	AI fallback path	Primary failure, valid secondary key	Secondary provider response	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Authentication Integration Test Cases is examined from the perspective of defect prevention, regression risk, and repeatability. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

AI and Fallback Test Cases

AI and Fallback Test Cases is examined from the perspective of verification planning and observable acceptance criteria. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass
TC-08	AI fallback path	Primary failure, valid secondary key	Secondary provider response	Pass
TC-09	Crop image required	Analyze without image	No fabricated result; upload warning	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

AI and Fallback Test Cases is examined from the perspective of defect prevention, regression risk, and repeatability. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Crop, Plant and Soil Test Cases

Crop, Plant and Soil Test Cases is examined from the perspective of verification planning and observable acceptance criteria. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass
TC-08	AI fallback path	Primary failure, valid secondary key	Secondary provider response	Pass
TC-09	Crop image required	Analyze without image	No fabricated result; upload warning	Pass
TC-10	Offline restoration	Network disabled after prior use	Cached shell and last snapshot shown	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Crop, Plant and Soil Test Cases is examined from the perspective of defect prevention, regression risk, and repeatability. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Weather and Scheme Test Cases

Weather and Scheme Test Cases is examined from the perspective of verification planning and observable acceptance criteria. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-01	Register valid farmer	Valid four-stage form	201 response; profile stored	Pass
TC-02	Reject invalid mobile	Nine digits or invalid prefix	Validation error; no insert	Pass
TC-03	Reject weak password	No letter or fewer than 8 chars	Password policy message	Pass
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Weather and Scheme Test Cases is examined from the perspective of defect prevention, regression risk, and repeatability. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Language, Voice and Accessibility Tests

Language, Voice and Accessibility Tests is examined from the perspective of verification planning and observable acceptance criteria. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-02	Reject invalid mobile	Nine digits or invalid prefix	Validation error; no insert	Pass
TC-03	Reject weak password	No letter or fewer than 8 chars	Password policy message	Pass
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Language, Voice and Accessibility Tests is examined from the perspective of defect prevention, regression risk, and repeatability. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Offline and Recovery Test Cases

Offline and Recovery Test Cases is examined from the perspective of verification planning and observable acceptance criteria. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-03	Reject weak password	No letter or fewer than 8 chars	Password policy message	Pass
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Offline and Recovery Test Cases is examined from the perspective of defect prevention, regression risk, and repeatability. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Compatibility and Responsive Tests

Compatibility and Responsive Tests is examined from the perspective of verification planning and observable acceptance criteria. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass
TC-08	AI fallback path	Primary failure, valid secondary key	Secondary provider response	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Compatibility and Responsive Tests is examined from the perspective of defect prevention, regression risk, and repeatability. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Security Test Cases

Security Test Cases is examined from the perspective of verification planning and observable acceptance criteria. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass
TC-08	AI fallback path	Primary failure, valid secondary key	Secondary provider response	Pass
TC-09	Crop image required	Analyze without image	No fabricated result; upload warning	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Security Test Cases is examined from the perspective of defect prevention, regression risk, and repeatability. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Defect Log and Corrective Actions

Defect Log and Corrective Actions is examined from the perspective of verification planning and observable acceptance criteria. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-06	Reject incorrect password	Correct mobile, wrong password	401 generic credential error	Pass
TC-07	AI primary success	Prompt and valid primary key	Localized KrishiBaba response	Pass
TC-08	AI fallback path	Primary failure, valid secondary key	Secondary provider response	Pass
TC-09	Crop image required	Analyze without image	No fabricated result; upload warning	Pass
TC-10	Offline restoration	Network disabled after prior use	Cached shell and last snapshot shown	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Defect Log and Corrective Actions is examined from the perspective of defect prevention, regression risk, and repeatability. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Test Summary and Acceptance Status

Test Summary and Acceptance Status is examined from the perspective of verification planning and observable acceptance criteria. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

ID	Scenario	Input / Condition	Expected Result	Status
TC-01	Register valid farmer	Valid four-stage form	201 response; profile stored	Pass
TC-02	Reject invalid mobile	Nine digits or invalid prefix	Validation error; no insert	Pass
TC-03	Reject weak password	No letter or fewer than 8 chars	Password policy message	Pass
TC-04	Prevent duplicate account	Existing mobile	409 conflict and login guidance	Pass
TC-05	Login valid account	Correct mobile and password	Profile returned; session persisted	Pass

Status interpretation: “Pass” records the manual or syntax-level acceptance observation available for the submitted prototype. The repository includes a comprehensive JavaScript syntax-check command, while a dedicated automated unit-test framework is not presently configured. This limitation is stated to avoid overstating the evidence.

Test Summary and Acceptance Status is examined from the perspective of defect prevention, regression risk, and repeatability. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Results and Discussion

Results and Discussion is examined from the perspective of problem context and academic relevance. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Results and Discussion is examined from the perspective of implemented behaviour and design evidence. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Results and Discussion is examined from the perspective of constraints, assumptions, and trade-offs. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Results and Discussion is examined from the perspective of measurement, acceptance, and future refinement. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Objective-Wise Evaluation

Objective-Wise Evaluation is examined from the perspective of problem context and academic relevance. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Objective-Wise Evaluation is examined from the perspective of implemented behaviour and design evidence. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Objective-Wise Evaluation is examined from the perspective of constraints, assumptions, and trade-offs. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Objective-Wise Evaluation is examined from the perspective of measurement, acceptance, and future refinement. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Performance and Reliability Discussion

Performance and Reliability Discussion is examined from the perspective of problem context and academic relevance. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Performance and Reliability Discussion is examined from the perspective of implemented behaviour and design evidence. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Performance and Reliability Discussion is examined from the perspective of constraints, assumptions, and trade-offs. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Performance and Reliability Discussion is examined from the perspective of measurement, acceptance, and future refinement. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Security and Privacy Discussion

Security and Privacy Discussion is examined from the perspective of problem context and academic relevance. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Security and Privacy Discussion is examined from the perspective of implemented behaviour and design evidence. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Security and Privacy Discussion is examined from the perspective of constraints, assumptions, and trade-offs. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Security and Privacy Discussion is examined from the perspective of measurement, acceptance, and future refinement. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Social Relevance and Ethical Considerations

Social Relevance and Ethical Considerations is examined from the perspective of problem context and academic relevance. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Social Relevance and Ethical Considerations is examined from the perspective of implemented behaviour and design evidence. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Social Relevance and Ethical Considerations is examined from the perspective of constraints, assumptions, and trade-offs. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Social Relevance and Ethical Considerations is examined from the perspective of measurement, acceptance, and future refinement. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Limitations

Limitations is examined from the perspective of problem context and academic relevance. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Limitations is examined from the perspective of implemented behaviour and design evidence. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Limitations is examined from the perspective of constraints, assumptions, and trade-offs. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Limitations is examined from the perspective of measurement, acceptance, and future refinement. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Future Scope

Future Scope is examined from the perspective of problem context and academic relevance. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Future Scope is examined from the perspective of implemented behaviour and design evidence. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Future Scope is examined from the perspective of constraints, assumptions, and trade-offs. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Future Scope is examined from the perspective of measurement, acceptance, and future refinement. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Maintenance and Support Plan

Maintenance and Support Plan is examined from the perspective of problem context and academic relevance. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Maintenance and Support Plan is examined from the perspective of implemented behaviour and design evidence. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Maintenance and Support Plan is examined from the perspective of constraints, assumptions, and trade-offs. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Maintenance and Support Plan is examined from the perspective of measurement, acceptance, and future refinement. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

User Manual: Getting Started

User Manual: Getting Started is examined from the perspective of step-by-step user operation. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

- Open the application and select a preferred language, or use Detect Language when location-based selection is desired.
- Register with valid identity and farm-profile values, then log in using the same ten-digit mobile number and password.
- Choose a dashboard service, provide the required question, location context, or image, and submit only after reviewing the input.
- Read or listen to the result, print or download drafts where offered, and treat health advice as decision support rather than a guaranteed diagnosis.
- When offline, consult the clearly marked last saved result. Reload after connectivity returns to request current information.

Common Mistakes

User Manual: Getting Started is examined from the perspective of common mistakes. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Privacy And Safety Guidance

User Manual: Getting Started is examined from the perspective of privacy and safety guidance. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Troubleshooting

User Manual: Getting Started is examined from the perspective of troubleshooting. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

User Manual: Advisory and Health Tools

User Manual: Advisory and Health Tools is examined from the perspective of step-by-step user operation. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

- Open the application and select a preferred language, or use Detect Language when location-based selection is desired.
- Register with valid identity and farm-profile values, then log in using the same ten-digit mobile number and password.
- Choose a dashboard service, provide the required question, location context, or image, and submit only after reviewing the input.
- Read or listen to the result, print or download drafts where offered, and treat health advice as decision support rather than a guaranteed diagnosis.
- When offline, consult the clearly marked last saved result. Reload after connectivity returns to request current information.

Common Mistakes

User Manual: Advisory and Health Tools is examined from the perspective of common mistakes. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Privacy And Safety Guidance

User Manual: Advisory and Health Tools is examined from the perspective of privacy and safety guidance. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Troubleshooting

User Manual: Advisory and Health Tools is examined from the perspective of troubleshooting. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

User Manual: Schemes, Drafts and Offline Use

User Manual: Schemes, Drafts and Offline Use is examined from the perspective of step-by-step user operation. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

- Open the application and select a preferred language, or use Detect Language when location-based selection is desired.
- Register with valid identity and farm-profile values, then log in using the same ten-digit mobile number and password.
- Choose a dashboard service, provide the required question, location context, or image, and submit only after reviewing the input.
- Read or listen to the result, print or download drafts where offered, and treat health advice as decision support rather than a guaranteed diagnosis.
- When offline, consult the clearly marked last saved result. Reload after connectivity returns to request current information.

Common Mistakes

User Manual: Schemes, Drafts and Offline Use is examined from the perspective of common mistakes. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Privacy And Safety Guidance

User Manual: Schemes, Drafts and Offline Use is examined from the perspective of privacy and safety guidance. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Troubleshooting

User Manual: Schemes, Drafts and Offline Use is examined from the perspective of troubleshooting. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Administrator and Deployment Manual

Administrator and Deployment Manual is examined from the perspective of step-by-step user operation. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

- Open the application and select a preferred language, or use Detect Language when location-based selection is desired.
- Register with valid identity and farm-profile values, then log in using the same ten-digit mobile number and password.
- Choose a dashboard service, provide the required question, location context, or image, and submit only after reviewing the input.
- Read or listen to the result, print or download drafts where offered, and treat health advice as decision support rather than a guaranteed diagnosis.
- When offline, consult the clearly marked last saved result. Reload after connectivity returns to request current information.

Common Mistakes

Administrator and Deployment Manual is examined from the perspective of common mistakes. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Privacy And Safety Guidance

Administrator and Deployment Manual is examined from the perspective of privacy and safety guidance. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Troubleshooting

Administrator and Deployment Manual is examined from the perspective of troubleshooting. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Conclusion

Conclusion is examined from the perspective of achievement against objectives. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Conclusion is examined from the perspective of technical learning. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Conclusion is examined from the perspective of practical value. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Conclusion is examined from the perspective of evidence and limitations. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Conclusion is examined from the perspective of final assessment. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Bibliography and Web References

Bibliography and Web References is examined from the perspective of achievement against objectives. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Bibliography and Web References is examined from the perspective of technical learning. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Bibliography and Web References is examined from the perspective of practical value. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Bibliography and Web References is examined from the perspective of evidence and limitations. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Bibliography and Web References is examined from the perspective of final assessment. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix A: API Contract Catalogue

Appendix A: API Contract Catalogue is examined from the perspective of problem context and academic relevance. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix A: API Contract Catalogue is examined from the perspective of implemented behaviour and design evidence. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix A: API Contract Catalogue is examined from the perspective of constraints, assumptions, and trade-offs. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Appendix A: API Contract Catalogue is examined from the perspective of measurement, acceptance, and future refinement. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix B: Environment and Deployment Checklist

Appendix B: Environment and Deployment Checklist is examined from the perspective of problem context and academic relevance. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix B: Environment and Deployment Checklist is examined from the perspective of implemented behaviour and design evidence. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix B: Environment and Deployment Checklist is examined from the perspective of constraints, assumptions, and trade-offs. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Appendix B: Environment and Deployment Checklist is examined from the perspective of measurement, acceptance, and future refinement. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix C: Sample Test Evidence Template

Appendix C: Sample Test Evidence Template is examined from the perspective of problem context and academic relevance. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix C: Sample Test Evidence Template is examined from the perspective of implemented behaviour and design evidence. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix C: Sample Test Evidence Template is examined from the perspective of constraints, assumptions, and trade-offs. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Appendix C: Sample Test Evidence Template is examined from the perspective of measurement, acceptance, and future refinement. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix D: Viva Questions and Model Answers

Appendix D: Viva Questions and Model Answers is examined from the perspective of problem context and academic relevance. Passwords are never stored in plain text. Registration generates a random salt and derives a 64-byte SHA-512 PBKDF2 hash using 120,000 iterations; login compares derived values with a timing-safe operation. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix D: Viva Questions and Model Answers is examined from the perspective of implemented behaviour and design evidence. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix D: Viva Questions and Model Answers is examined from the perspective of constraints, assumptions, and trade-offs. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Appendix D: Viva Questions and Model Answers is examined from the perspective of measurement, acceptance, and future refinement. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix E: Glossary and Index

Appendix E: Glossary and Index is examined from the perspective of problem context and academic relevance. The service worker caches the application shell while browser storage retains the authenticated state, manual language choice, AI cache, and last successful advisory snapshots for reference during network loss. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix E: Glossary and Index is examined from the perspective of implemented behaviour and design evidence. The AI endpoint presents the model as KrishiBaba and calls a primary Groq credential first. A second independently configured Groq credential and model are attempted when the primary provider fails. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. Static translations and dynamic localization support English, Hindi, Bhojpuri, Gujarati, Marathi, Kannada, Tamil, Telugu, Punjabi, and Haryanvi. A manual selection remains persistent until language detection is explicitly requested. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Appendix E: Glossary and Index is examined from the perspective of constraints, assumptions, and trade-offs. Crop and plant image analysis is proxied through server-side routes so provider credentials are not exposed to browser JavaScript. Crop Kindwise and Plant.id responses are converted into farmer-friendly treatment guidance. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. KrishiGyaan is a responsive agricultural support platform implemented with static HTML, CSS and vanilla JavaScript, Node.js serverless API routes, MongoDB Atlas, and third-party agricultural services. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.

Dimension	Project-Specific Observation
User value	Consolidates fragmented agricultural information into one localized dashboard.
Technical value	Demonstrates frontend, serverless backend, database, API, offline, security, and deployment skills.
Constraint	AI and image results depend on connectivity, provider availability, quota, and input quality.
Control	Validation, fallback, offline snapshots, clear error messages, and protected credentials reduce operational risk.

Appendix E: Glossary and Index is examined from the perspective of measurement, acceptance, and future refinement. Open-Meteo forecast and reverse-geocoding services provide rainfall, temperature, wind, location, and planning context without placing a proprietary weather key in the browser. In the implemented system, this concern is not isolated: it influences user trust, response clarity, fault handling, and the ability to use the platform under real farm conditions. The principal user is an Indian farmer who may have limited digital literacy, intermittent connectivity, a preference for a regional language, and a need for concise decisions rather than technical raw data. The design decision is therefore documented as a verifiable project requirement rather than an informal feature claim. Acceptance is based on observable input, processing, output, and failure behaviour, with assumptions stated wherever an external service controls the final response.